A. Multithreading in Java — 15 Questions

1. What is a thread in Java?

A thread is an independent path of execution within a program. A Java application starts with at least one thread, commonly called the main thread. A program can create additional threads to perform other work.

Threads in the same process share memory, including objects in the heap, but each thread has its own call stack.

```
Thread worker = new Thread(() -> System.out.println("Working"));

worker.start();
```

2. What is multithreading?

Multithreading means running multiple threads within one program. The threads may make progress concurrently: their work can overlap, even if the computer switches between them rather than executing them at precisely the same instant.

```
Thread first = new Thread(() -> System.out.println("First task"));

Thread second = new Thread(() -> System.out.println("Second task"));

first.start();

second.start();
```

3. Why is multithreading used in Java applications?

Multithreading can help an application:

- Keep its user interface responsive while work runs in the background.
- Handle multiple requests or tasks at the same time.
- Overlap waiting for input/output, such as network or file operations.
- Make use of multiple processor cores for suitable CPU-intensive work.

Threads also introduce challenges, such as coordinating access to shared data and handling failures. Creating a thread for every task is often not the best approach; Java's executor services can manage groups of tasks and threads.

4. What is the difference between a process and a thread?

Feature	Process	Thread
Meaning	A running program with its own resources	A path of execution within a process
Memory	Usually has its own separate address space	Shares the process's memory with other threads

Feature	Process	Thread
Communication	Often requires operating-system mechanisms	Can communicate through shared objects
Resource use	Generally heavier to create and manage	Generally lighter than a process
Failure impact	A process failure is usually isolated from other processes	An unhandled thread failure can affect its application, depending on the situation

Threads share memory, which makes communication convenient, but it also means access to shared data may need coordination.

5. How can you create a thread in Java?

Two traditional ways are to implement Runnable or extend Thread.

Implement Runnable:

```
Runnable task = () -> System.out.println("Task is running");
Thread thread = new Thread(task);
thread.start();
```

Extend Thread:

```
class WorkerThread extends Thread {
  @Override
  public void run() {
    System.out.println("Task is running");
  }
}
new WorkerThread().start();
```

6. What is the difference between extending Thread and implementing Runnable?

Feature	Extend Thread	Implement Runnable
What you define	A thread class	A task that can run on a thread
Java inheritance	Uses the class's one allowed superclass	Leaves the class free to extend another class
Reuse	Task and thread are combined in one class	Task can be run by different threads or an executor

Feature	Extend Thread	Implement Runnable
Typical use	Simple examples or specialized thread behavior	A common choice for separating work from thread management

Runnable is generally more flexible because it separates the task from the mechanism that runs it.

7. What is the purpose of the start() method?

start() asks the JVM to start a new thread of execution. The JVM then invokes that thread's run() method.

```
Thread thread = new Thread(() -> System.out.println("Runs on a new thread"));

thread.start();
```

A Thread object can be started only once. Calling start() on it again throws IllegalThreadStateException.

8. What is the difference between start() and run()?

Feature	start()	run()
Starts a new thread?	Yes	No
How the code runs	The JVM starts a new thread and invokes run() on it	Runs like an ordinary method on the current thread
Call it when	You want concurrent execution	You want to call the method directly

```
Thread thread = new Thread(() ->
   System.out.println(Thread.currentThread().getName())
);

thread.start(); // Runs on a new thread

thread.run();   // Runs on the current thread
```

Calling run() directly does not start the thread.

9. What is the life cycle of a thread?

Java defines these thread states in Thread.State:

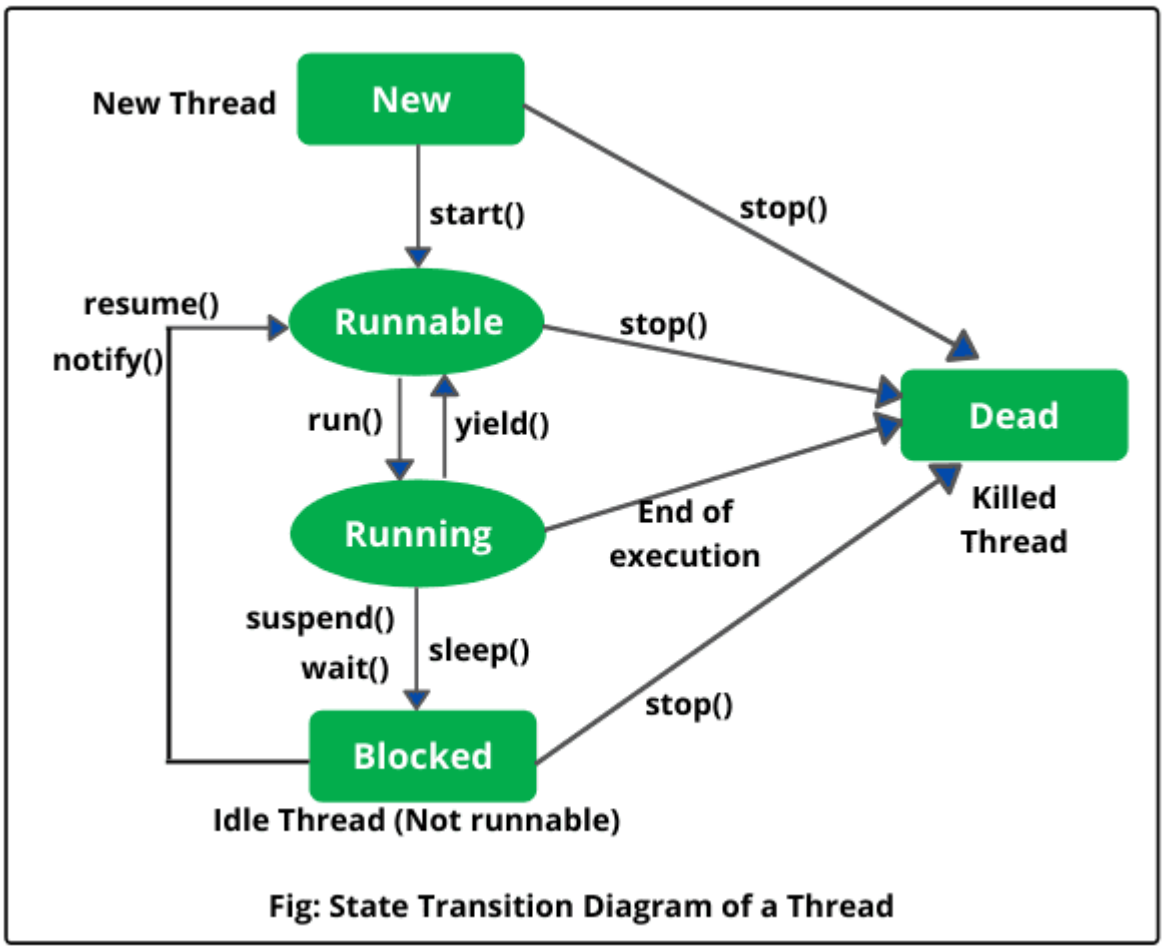


Fig: State Transition Diagram of a Thread

State in the diagram	Meaning	How a thread enters this state	How it moves on
New	The thread object has been created, but it has not started executing.	Create a Thread object.	Calling start() moves it to **Runnable**.
Runnable	The thread is ready to run and waiting for CPU time.	Call start(), or the thread becomes ready after waiting or blocking.	The thread scheduler selects it to run.
Running	The thread is executing its run() method.	The scheduler selects the thread from **Runnable**.	It may finish and become **Dead**, or pause and return to **Runnable**.
Blocked / Waiting	The thread cannot continue temporarily—for example, it is waiting for a lock, waiting for	It may try to acquire a lock held by another	After the lock becomes available, it is notified, or its sleep time ends, it

State in the diagram	Meaning	How a thread enters this state	How it moves on
	another thread, or sleeping.	thread, call wait(), or call sleep().	can return to **Runnable**.
Dead / Terminated	The thread has finished execution.	Its run() method completes.	This is the end of the thread's life cycle; it cannot be started again.

These are the states exposed by Java; they are not a simple sequence that every thread visits in order.

10. What is the purpose of sleep()?

Thread.sleep() pauses the current thread for approximately the specified time. It does not release any locks that the thread holds.

```
try {
   Thread.sleep(1000); // pause for about one second
} catch (InterruptedException e) {
   Thread.currentThread().interrupt();
}
```

The delay is not a guarantee that the thread will resume at exactly that moment. Other scheduling work can delay it. sleep() can throw InterruptedException, so code should handle interruption appropriately.

11. What is the difference between sleep() and wait()?

Feature	sleep()	wait()
Defined on	Thread	Object
Main purpose	Pause a thread for a period of time	Wait for a notification or a condition involving an object
Releases a monitor lock?	No	Yes, releases the monitor for the object on which it is called
Must hold a monitor?	No	Yes; otherwise, IllegalMonitorStateException is thrown
How it resumes	After the time passes, subject to scheduling and interruption	After notification, timeout, or interruption; it must reacquire the monitor before continuing

A call to wait() must be made while synchronized on the same object:

```
synchronized (lock) {
  lock.wait();
}
```

12. What is thread synchronization?

Thread synchronization coordinates access to shared data so that threads do not interfere with one another. It can prevent certain race conditions and ensure that changes made by one thread become visible to others under the Java Memory Model.

Java provides several synchronization tools, including:

- The synchronized keyword and object monitors.
- Lock implementations such as ReentrantLock.
- Atomic classes such as AtomicInteger.
- Concurrent collections and other classes in java.util.concurrent.

Synchronization should protect the shared operation or state that needs coordination. It can also affect performance and cause deadlocks if locks are used carelessly.

13. Why is the synchronized keyword used?

synchronized controls access to a block or method using a monitor lock. Only one thread at a time can hold a particular monitor.

```
class Counter {
  private int count;
    public synchronized void increment() {
    count++;
  }
  public synchronized int getCount() {
  return count;
  }
}
```

For an instance method, the monitor is the current object. For a static synchronized method, it is the class's Class object. Synchronization also provides visibility guarantees: when one thread releases a monitor, a later thread acquiring that same monitor can see the changes protected by it.

14. What is a race condition?

A race condition happens when a program's result depends on the unpredictable timing or interleaving of multiple threads accessing shared state.

For example, count++ is not one indivisible operation. It involves reading the value, adding one, and writing it back. Two threads can interleave those steps and lose an update.

```
class Counter {
  int count = 0;
  void increment() {
      count++; // unsafe if called concurrently without coordination
  }
}
```

15. What is the difference between join() and sleep()?

Feature	join()	sleep()
Purpose	Wait for a particular thread to finish	Pause the current thread for a time
Called on	The thread to wait for, such as worker.join()	Thread.sleep(...)
How long it waits	Until the target thread finishes, or until an optional timeout	For the specified time, subject to scheduling
Lock behavior	Does not release locks held by the calling thread	Does not release locks held by the calling thread
Can be interrupted?	Yes; it throws InterruptedException	Yes; it throws InterruptedException

```
Thread worker = new Thread(() -> System.out.println("Work complete"));
worker.start();
try {
  worker.join(); // main thread waits for worker to finish
} catch (InterruptedException e) {
  Thread.currentThread().interrupt();
}
```

B. File Handling in Java — 15 Questions

16. What is file handling in Java?

File handling means creating, reading, writing, updating, and managing files and directories from a Java program.

Java offers several APIs for this:

- java.io provides streams, readers, writers, and the older File class.
- java.nio.file provides the newer Path and Files APIs.

17. Which package is commonly used for file handling in Java?

Two commonly used packages are:

- java.io — includes File, FileReader, FileWriter, FileInputStream, FileOutputStream, BufferedReader, and related classes.
- java.nio.file — includes Path and Files, which provide modern file and directory operations.

For example, Files.readString() and Files.writeString() are convenient for basic text-file tasks in modern Java versions.

```
import java.nio.file.Files;

import java.nio.file.Path;
```

18. What is the purpose of the File class?

java.io.File represents a file-system path. It provides methods for checking or manipulating path-related information, such as whether a path exists, its name, or whether it refers to a directory.

```
File file = new File("notes.txt");

System.out.println(file.exists());

System.out.println(file.getName());

System.out.println(file.isDirectory());
```

Creating a File object does not create a file on disk. The class is also considered a legacy API for many file operations; modern code often uses Path and Files.

19. How do you create a new file using Java?

With the File class, call createNewFile(). It returns true if a new file was created and false if a file already exists at that path. It can throw IOException.

```
import java.io.File;

import java.io.IOException;

File file = new File("notes.txt");

try {

  if (file.createNewFile()) {

    System.out.println("File created");

  } else {

    System.out.println("File already exists");
```

```
    }
} catch (IOException e) {
    System.err.println("Could not create file: " + e.getMessage());
}
```

A modern alternative is Files.createFile(Path), which throws FileAlreadyExistsException if the file already exists.

20. How do you check whether a file exists?

Using the older File API:

```
File file = new File("notes.txt");
if (file.exists()) {
    System.out.println("The path exists");
}
```

Using NIO.2:

```
Path path = Path.of("notes.txt");
if (Files.exists(path)) {
    System.out.println("The path exists");
}
```

21. How do you obtain the name and path of a file?

With File:

```
File file = new File("docs/notes.txt");
System.out.println(file.getName());      // notes.txt
System.out.println(file.getPath());      // path as supplied
System.out.println(file.getAbsolutePath());// absolute path
```

With Path:

```
Path path = Path.of("docs", "notes.txt");
System.out.println(path.getFileName()); // notes.txt
System.out.println(path.toAbsolutePath());
```

An absolute path identifies a location from the file system's root. A relative path is interpreted relative to the program's working directory.

22. What is the difference between FileReader and FileWriter?

Feature	FileReader	FileWriter
Purpose	Reads text from a file	Writes text to a file
Direction	File → program	Program → file
Data type	Characters	Characters
Common methods	read()	write()
Typical use	Reading character text	Writing character text
Resource handling	Should be closed, preferably with try-with-resources	Should be closed, preferably with try-with-resources

```
try (FileReader reader = new FileReader("input.txt");
    FileWriter writer = new FileWriter("output.txt")) {
   // Read from reader and write to writer
}
```

23. What is the difference between character streams and byte streams?

Feature	Character streams	Byte streams
Data handled	Text characters	Raw bytes
Base classes	Reader and Writer	InputStream and OutputStream
Encoding	Convert between bytes and characters using a character encoding	Do not themselves interpret bytes as text
Common classes	FileReader, BufferedReader, FileWriter	FileInputStream, BufferedInputStream, FileOutputStream
Typical use	Text files	Images, audio, PDFs, and other binary files

24. What is the purpose of FileInputStream?

FileInputStream reads raw bytes from a file. It is appropriate for binary data and can also be used as a lower-level source when reading bytes directly.

```
try (FileInputStream input = new FileInputStream("image.png")) {
   int oneByte = input.read();
```

```
    System.out.println(oneByte);
} catch (IOException e) {
    System.err.println("Could not read file: " + e.getMessage());
}
```

For more efficient reading, it can be wrapped in a BufferedInputStream. For text, prefer a character-oriented reader that handles decoding.

25. What is the purpose of FileOutputStream?

FileOutputStream writes raw bytes to a file. It is commonly used for binary data.

```
byte[] data = {65, 66, 67};
try (FileOutputStream output = new FileOutputStream("data.bin")) {
    output.write(data);
} catch (IOException e) {
    System.err.println("Could not write file: " + e.getMessage());
}
```

By default, opening a FileOutputStream for an existing file truncates it. To append instead, use the constructor with true:

```
new FileOutputStream("data.bin", true);
```

26. What is the difference between BufferedReader and FileReader?

Feature	FileReader	BufferedReader
Role	Reads characters from a file	Adds buffering around another Reader
Reads lines directly?	No readLine() method	Provides readLine()
Buffering	Does not provide the same explicit line-reading convenience	Reads data into a buffer, reducing frequent low-level reads
How commonly used	Can be wrapped by a BufferedReader	Often used around a file reader for efficient text reading

```
try (BufferedReader reader =
        new BufferedReader(new FileReader("notes.txt"))) {
    String line = reader.readLine();
}
```

For explicit encoding and modern file handling, Files.newBufferedReader(path, charset) is a useful alternative.

27. How do you read a file line by line in Java?

Use a BufferedReader and repeatedly call readLine(). It returns null at the end of the file.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
Path path = Path.of("notes.txt");
try (BufferedReader reader =
      Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
  String line;
  while ((line = reader.readLine()) != null) {
    System.out.println(line);
  }
} catch (IOException e) {
  System.err.println("Could not read file: " + e.getMessage());
}
```

28. How do you write data into a text file?

For a small amount of text, Files.writeString() is a convenient option:

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
Path path = Path.of("output.txt");
try {
  Files.writeString(path, "Hello, Java!\n", StandardCharsets.UTF_8);
} catch (IOException e) {
  System.err.println("Could not write file: " + e.getMessage());
}
```

This replaces the file's existing contents by default. To append, specify an append option:

```
Files.writeString(
  path,
  "Another line\n",
  StandardCharsets.UTF_8,
  StandardOpenOption.CREATE,
  StandardOpenOption.APPEND
);
```

29. Why should files be closed after reading or writing?

Files should be closed so that the program releases operating-system resources, such as file handles, and ensures buffered output is written. Leaving files open can prevent other code or programs from accessing them as expected and can exhaust available resources.

Use try-with-resources to close them automatically:

```
try (BufferedWriter writer = Files.newBufferedWriter(Path.of("output.txt"))) {
  writer.write("Saved text");
} catch (IOException e) {
  System.err.println("File operation failed: " + e.getMessage());
}
```

The resource is closed when the block exits, whether it completes normally or an exception occurs.

30. What exceptions can occur during file handling, and how can they be handled?

File operations can fail for different reasons. Common exceptions include:

Exception	Possible cause
IOException	General input/output failure
FileNotFoundException	A file cannot be opened, often because it does not exist or access is denied
NoSuchFileException	A path does not exist when an NIO file operation expects it
AccessDeniedException	The program does not have permission for the operation
FileAlreadyExistsException	A create operation targets an existing path
SecurityException	A security manager or other security check denies access

Catch a specific exception when you can handle it differently; otherwise, catch IOException for general file-operation failures. Try-with-resources handles closing the file while catch handles the error:

```
Path path = Path.of("input.txt");

try {
   String content = Files.readString(path);
   System.out.println(content);
} catch (NoSuchFileException e) {
   System.err.println("The file does not exist: " + path);
} catch (IOException e) {
   System.err.println("Could not read the file: " + e.getMessage());
}
```

Handle errors meaningfully—for example, report which file failed or choose a recovery action—instead of silently ignoring exceptions.